Software Requirements Specification

for

<Singapore Zone-Based Real Estate Linear Regression Predictor>

Version 1.0 approved

Prepared by

<author>

<Group 2>

<date created>

Copyright © 1999 by Karl E. Wiegers. Permission is granted to use, modify, and distribute this document.

Table of Contents

1. Introduction **1**
1.1 Purpose 1
1.2 Document Conventions **1**
1.3 Intended Audience and Reading Suggestions **1**
<Provide a short description of the software being specified and its purpose, including relevant benefits, objectives, and goals. Relate the software to corporate goals or business strategies. If a separate vision and scope document is available, refer to it rather than duplicating its contents here.> **1**
1.5 References 2
2. Overall Description **2**
2.1 Product Perspective 2
2.2 Product Functions **3**
2.3 User Classes and Characteristics **4**
2.4 Operating Environment **4**
2.5 Design and Implementation Constraints **5**
2.6 User Documentation **5**
2.7 Assumptions and Dependencies **5**
3. External Interface Requirements **6**
3.1 User Interfaces 6
3.2 Hardware Interfaces **7**
3.3 Software Interfaces **7**
3.4 Communications Interfaces **8**
4. System Features **8**
4.1 Authentication 8
4.2 Housing Search 9
4.3 Historical Data 9
4.4 Price Analysis 10
4.5 Area Comparison 10
4.6 Price Prediction 11
4.7 Model Diagnostics 11
4.8 Validation and Error Handling 12
4.9 Dataset Management 12
5. Other Nonfunctional Requirements **13**
5.1 Performance Requirements 13
5.2 Safety Requirements **13**
5.3 Security Requirements **14**
5.4 Software Quality Attributes **14**
5.5 Business Rules **15**
6. Other Requirements **15**
6.1 Data Requirements 15
6.2 Forecast Output Requirements 15

Revision History

Name	Date	Reason For Changes	Version

1. Introduction

1.1 Purpose

This Software Requirements Specification (SRS) defines the functional and non-functional requirements for Version 1.0 of the ***Singapore Zone-Based Real Estate Linear Regression Predictor****. The core purpose of this software is to estimate real estate resale prices across different zones in Singapore using an Ordinary Least Squares (OLS) Multiple Linear Regression model built upon structured historical transaction datasets.*

Current Estimator of house pricing. Purpose: For users to find the prices of houses based on historical data.

1.2 Document Conventions

- *Functional requirements are identified using the notation FR-XX.*
- *Non-functional requirements are identified using the notation NFR-XX.*
- *The term "shall" denotes a mandatory system requirement.*
- *Monetary values are expressed in Singapore Dollars (SGD).*
- *Forecast periods are expressed in years.*
- *Predicted resale price may be represented as Y-hat (Ŷ) in technical model descriptions.*

Mathematical Notation*: Regression equations follow standard matrix notation* $\hat{Y} = X\beta + \varepsilon$.

1.3 Intended Audience and Reading Suggestions

The system ingests tabular real estate records comprising categorical variables (town, flat_type, storey_range, flat_model) and continuous variables (floor_area_sqm, lease_commence_date, remaining_lease). It applies one-hot/dummy variable encoding to construct a numerical design matrix X and executes a Multiple Linear Regression model to forecast the continuous target variable Y (resale_price).

1.4 Product Scope

Data Scientists & Machine Learning Engineers*: Focus on Section 4.1, Section 4.2, and Appendix B for encoding, variable selection, and matrix operations.*

Full-Stack Developers*: Focus on Section 3 (Interfaces) and Section 4.3 (User Parameters).*

Project Managers & Domain Analysts*: Review Sections 2, 4.4, and 5 for system functionality and performance guarantees.*

Refer to Table of Content

1.5 References (APA)

1. *CE2006/CZ2006/CPE207/CSC207 Software Engineering Laboratory Manual - Overview.*
2. *CE2006/CZ2006/CPE207/CSC207 Lab #1: Requirements Elicitation.*
3. *HDB resale transaction dataset - exact dataset, version, and source to be confirmed by the development team.*

2. Overall Description

2.1 Product Perspective

The software functions as an end-to-end analytical web application. The backend houses a Python-based regression module executing numerical matrix calculations via `scikit-learn` / `statsmodels` and serves linear estimates over a REST API to a web UI.

****
The system is a new, self-contained web application consisting of four major components:

User Interface - *allows users to select towns and housing characteristics and view historical and forecast results.*

Data Processing Component - *validates, cleans, filters, and transforms historical resale transaction data.*

Analytical and Forecasting Component - *analyses historical price data and generates future resale-price estimates.*

Data Storage Component - *stores historical housing transaction records and data required by the system.*

The user interacts with the system through a web browser. User selections are processed by the backend, which retrieves relevant historical data, performs the requested calculations, and returns results to the web interface.

2.2 Product Functions

Dataset Ingestion & Validation*: Ingest .csv / .xlsx real estate records matching the official HDB tabular schema.*

Feature Encoding & Preprocessing*: Convert categorical variables (town, storey_range, flat_type, flat_model) into one-hot dummy columns while preventing multi-collinearity.*

OLS Coefficient Computation*: Train and compute coefficient weights ($\beta_0, \beta_1, \dots, \beta_p$) across all independent predictors.*

Real-Time Linear Valuation*: Calculate instantaneous property predictions ($\hat{Y}$) based on linear combinations of user-selected inputs.*

Model Diagnostics Output*: Report goodness-of-fit metrics (R^2, Adjusted R^2, Root Mean Squared Error).*

2.3 User Classes and Characteristics

General User

The General User is the primary user of the system. This may include prospective HDB buyers, existing HDB owners, students, researchers, or members of the public interested in understanding HDB resale-price trends. No specialised knowledge of statistics, programming, or machine learning is required. Users interact with the system through graphical controls such as dropdown menus, filters, charts, and tables.

Administrator

The Administrator will be responsible for maintaining the housing dataset used by the system. The administrator may import, validate, or replace housing transaction datasets.

2.4 Operating Environment

<Describe the environment in which the software will operate, including the hardware platform, operating system and versions, and any other software components or applications with which it must peacefully coexist.>

***Backend Runtime**: Python 3.11+ running NumPy, Pandas, Scikit-Learn, Statsmodels.*

***Frontend Runtime**: Modern Browser Engine (HTML5/CSS3/JavaScript, React/Vue framework).*

***Database / File Storage**: PostgreSQL / SQLite or local `.csv` transaction store.*

2.5 Design and Implementation Constraints

<Describe any items or issues that will limit the options available to the developers. These might include: corporate or regulatory policies; hardware limitations (timing requirements, memory requirements); interfaces to other applications; specific technologies, tools, and databases to be used; parallel operations; language requirements; communications protocols; security considerations; design conventions or programming standards (for example, if the customer's organization will be responsible for maintaining the delivered software).>

***Linearity Assumption**: Predictions rely strictly on Multiple Linear Regression without non-linear tree-based ensembles (such as XGBoost or Random Forests).*
***Multicollinearity Prevention**: The feature matrix pipeline must handle high collinearity between `lease_commence_date` and `remaining_lease` (e.g., via Variance Inflation Factor analysis or feature selection).*

***Deterministic Inference**: Inference output must execute as a fast dot product vector operation $\hat{y} = \vec{x} \cdot \vec{\beta}$.*

2.6 User Documentation

<List the user documentation components (such as user manuals, on-line help, and tutorials) that will be delivered along with the software. Identify any known user documentation delivery formats or standards.>

***User Manual**: Documentation detailing the functions of user interfaces of the website.*

2.7 Assumptions and Dependencies

<List any assumed factors (as opposed to known facts) that could affect the requirements stated in the SRS. These could include third-party or commercial components that you plan to use, issues around the development or operating environment, or constraints. The project could be affected if these assumptions are incorrect, are not shared, or change. Also identify any dependencies the project has on external factors, such as software components that you intend to reuse from another project, unless they are already documented elsewhere (for example, in the vision and scope document or the project plan).>

The relationship between `floor_area_sqm` and `resale_price` is predominantly linear within specific zone categories.

Uploaded spreadsheet files contain mandatory columns: `town`, `flat_type`, `storey_range`, `floor_area_sqm`, `lease_commence_date`, `remaining_lease`, and `resale_price`.

***A-01:** Historical HDB resale transaction data is sufficiently complete for the selected towns and flat types.*

***A-02:** The historical dataset contains the attributes required for analysis.*

***A-03:** Historical transaction records include a transaction period or another time-related variable from which price trends can be calculated.*

***A-04:** Historical patterns provide some useful information for estimating future resale-price trends.*

***A-05:** Forecast accuracy generally decreases as the forecasting horizon increases.*

***A-06:** Users understand that forecast values are estimates rather than guaranteed future transaction prices.*

D-01: *The system depends on the availability of the selected historical HDB resale transaction dataset.*

D-02: *The system depends on the selected numerical and statistical software libraries.*

D-03: *Data preprocessing and forecasting components must use compatible data formats.*

3. External Interface Requirements

3.1 User Interfaces

<Describe the logical characteristics of each interface between the software product and the users. This may include sample screen images, any GUI standards or product family style guides that are to be followed, screen layout constraints, standard buttons and functions (e.g., help) that will appear on every screen, keyboard shortcuts, error message display standards, and so on. Define the software components for which a user interface is needed. Details of the user interface design should be documented in a separate user interface specification.>

Parameter Input Panel*: Dropdowns for town, flat_type, storey_range, and flat_model, plus numerical input fields for floor_area_sqm and lease_commence_date.*

Valuation Display*: Prominent text block rendering predicted price ($\hat{Y}$ in SGD) and residual variance bound ($\pm \text{RMSE}$).*

Linear Coefficient Breakdown Table*: Table displaying the linear contribution of each selected property feature to the final price.*

Home/Search Interface: *Allows the user to select a Singapore town and relevant HDB housing characteristics.*

Town Analysis Interface: *Displays historical resale-price statistics and historical price trends for the selected town.*

Comparison Interface: *Allows users to select two or more towns and compare historical price trends.*

Forecast Interface: *Allows the user to select a forecast period from One to Ten years and displays estimated future resale-price values.*

Transaction Interface: *Displays historical transaction records matching user-selected filters.*

3.2 Hardware Interfaces

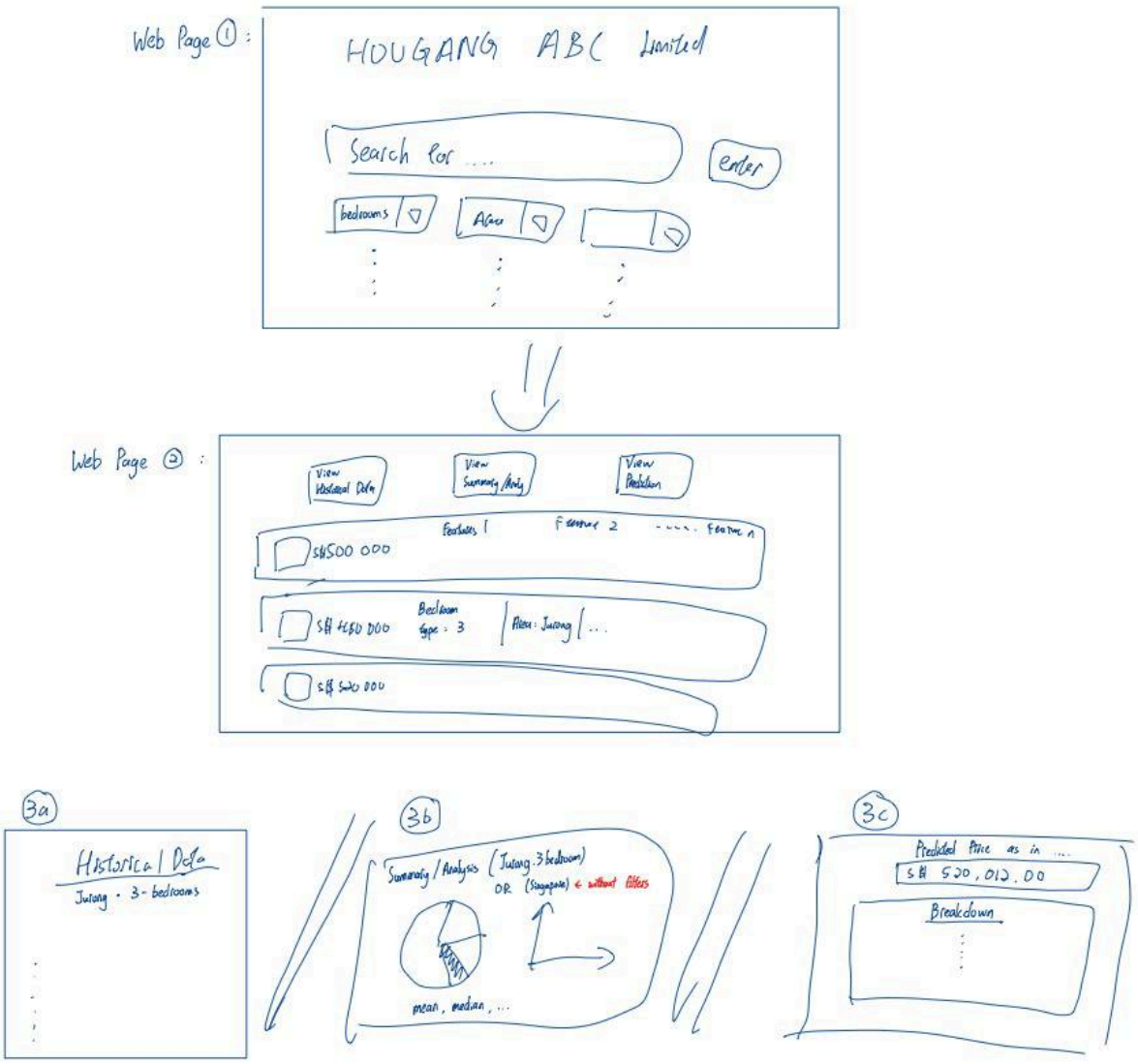
<Describe the logical and physical characteristics of each interface between the software product and the hardware components of the system. This may include the supported device types, the nature of the data and control interactions between the software and the hardware, and communication protocols to be used.>

Standard cloud host virtual machines (x86-64 / ARM64 architecture, min 2 GB RAM).

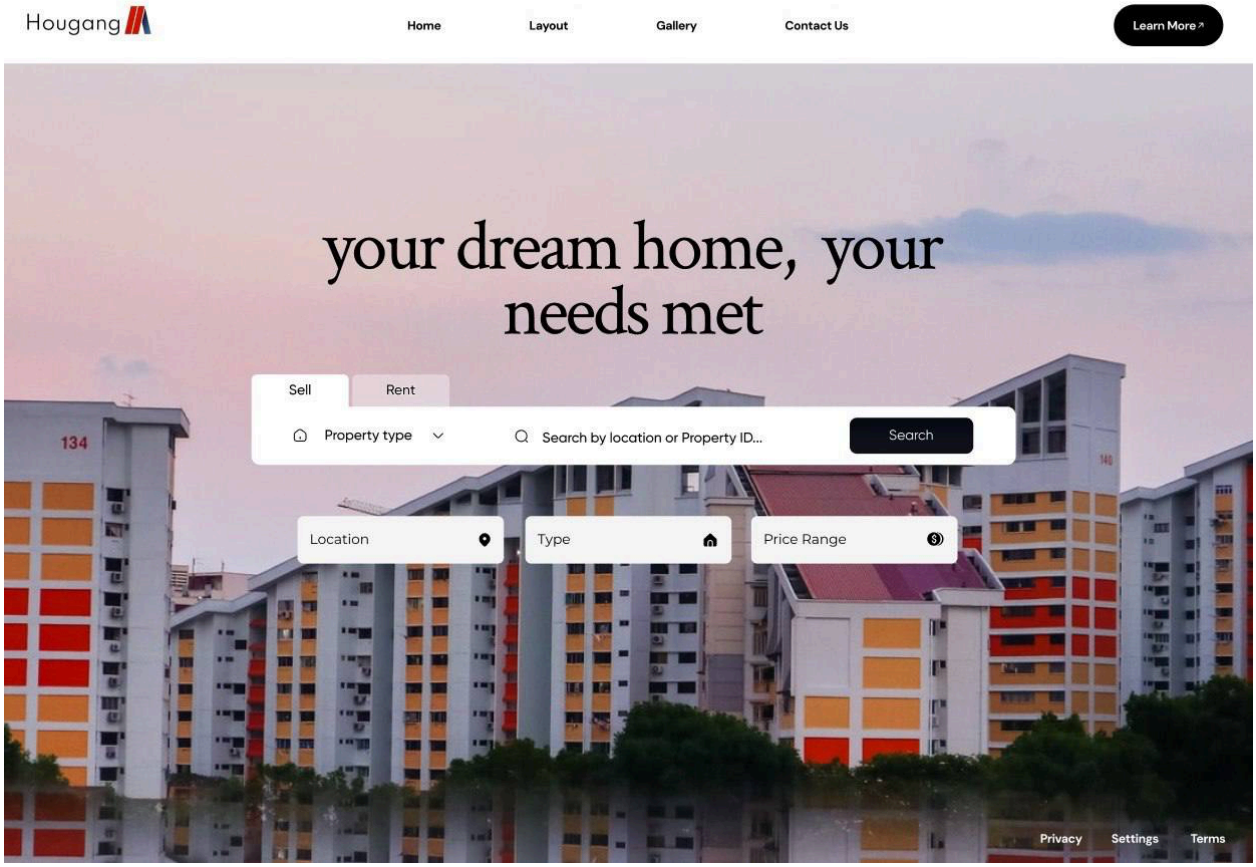
3.3 Software Interfaces (name all UI page, same name as code)

<Describe the connections between this product and other specific software components (name and version), including databases, operating systems, tools, libraries, and integrated commercial components. Identify the data items or messages coming into the system and going out and describe the purpose of each. Describe the services needed and the nature of communications. Refer to documents that describe detailed application programming interface protocols. Identify data that will be shared across software components. If the data sharing mechanism must be implemented in a specific way (for example, use of a global data area in a multitasking operating system), specify this as an implementation constraint.>

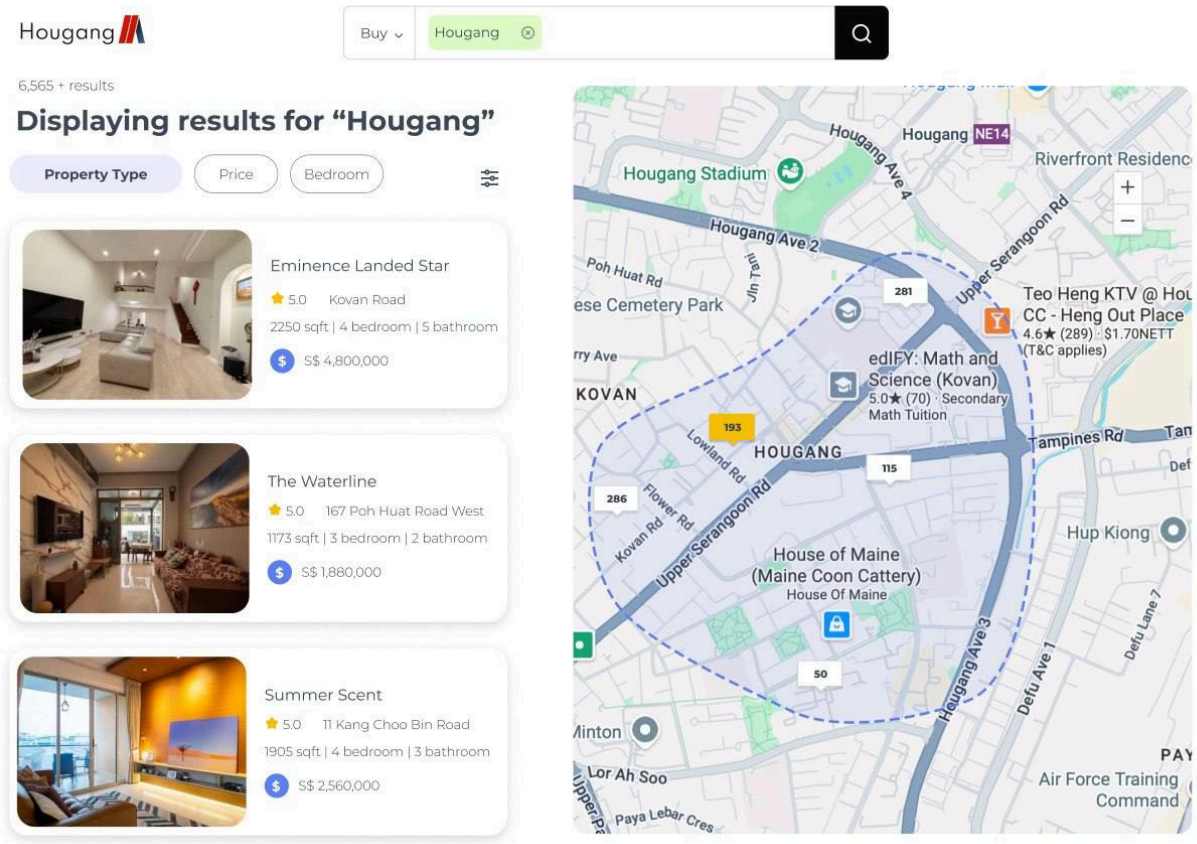
***Python NumPy/Scikit-Learn API**: Invoked to perform vector arithmetic:*
$$\hat{Y} = \beta_0 + \sum_{i=1}^{n} \beta_i X_i$$
***Spreadsheet Ingestion Interface**: Native pandas read_csv / read_excel ingestion routines.*



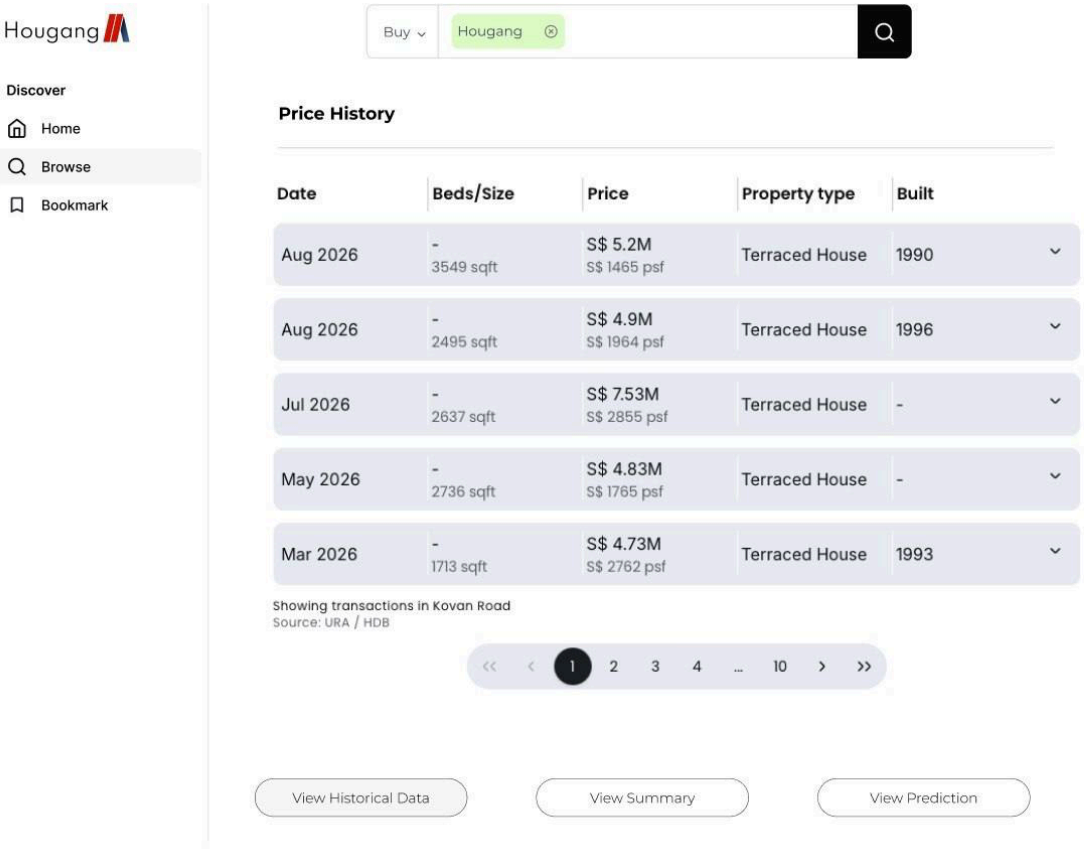
Label the figures



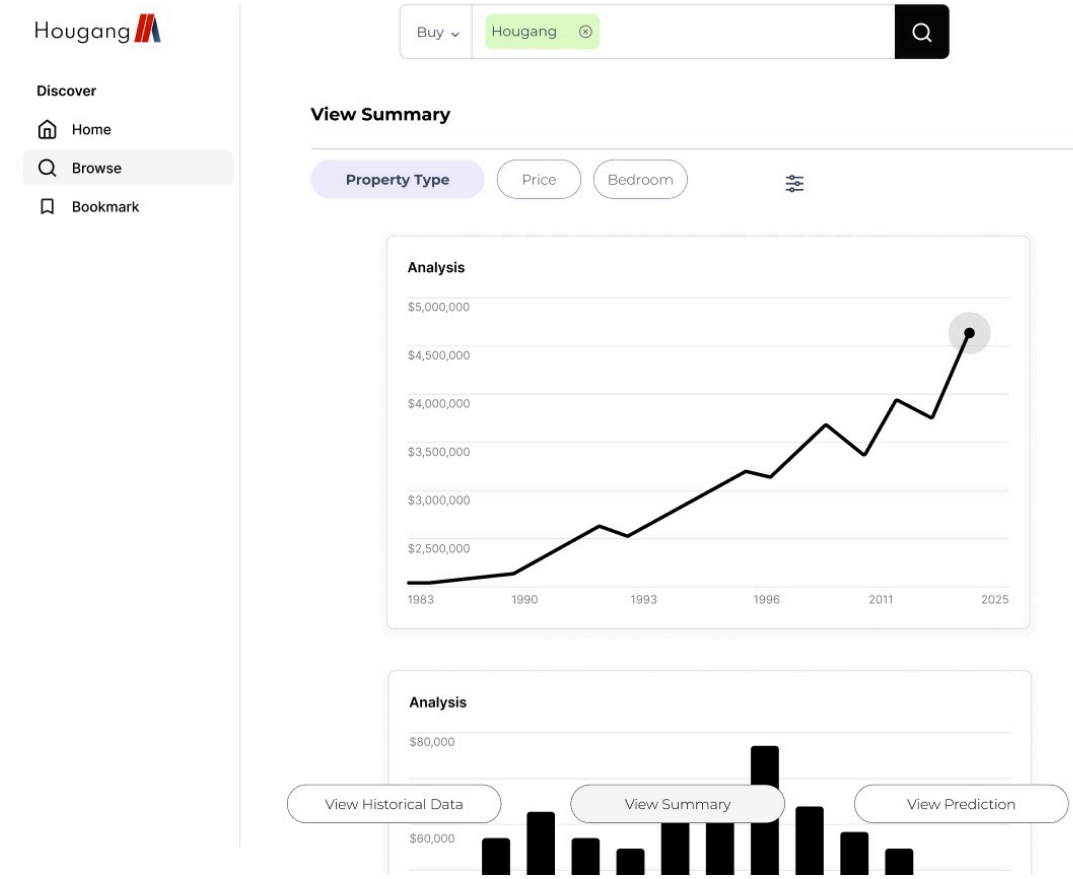
Label the figures



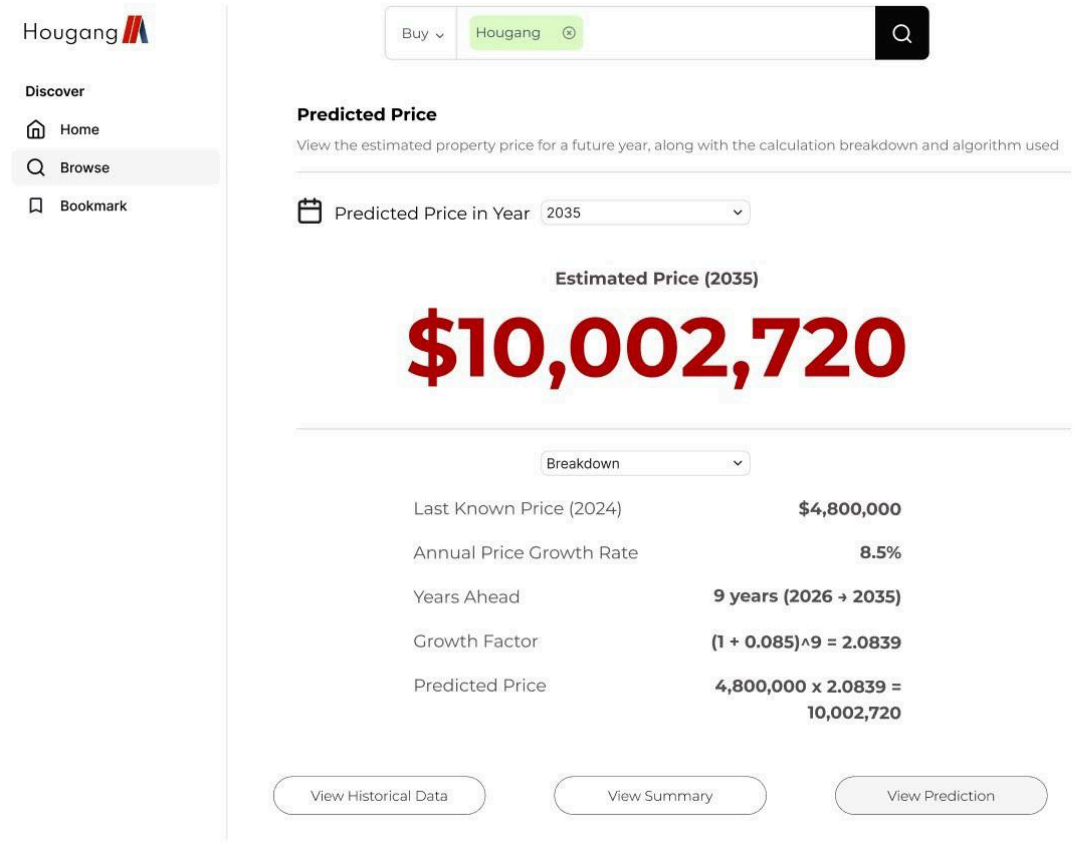
Label the figures



Label the figures



Label the figures



Label the figures

3.4 Communications Interfaces

<Describe the requirements associated with any communications functions required by this product, including e-mail, web browser, network server communications protocols, electronic forms, and so on. Define any pertinent message formatting. Identify any communication standards that will be used, such as FTP or HTTP. Specify any communication security or encryption issues, data transfer rates, and synchronization mechanisms.>

HTTPS communication over TLS 1.3 using JSON payloads for request features and response valuations.

4. System Features

<This template illustrates organizing the functional requirements for the product by system features, the major services provided by the product. You may prefer to organize this section by

use case, mode of operation, user class, object class, functional hierarchy, or combinations of these, whatever makes the most logical sense for your product.>

4.1 Select Housing Filters (End User)

4.1.1 Description and Priority

Lets the user narrow the transaction dataset to a specific town, flat type, and additional housing characteristics as the basis for all downstream analysis and prediction.

***Priority**: High.*

4.1.2 Stimulus/Response Sequences

1. *User selects a town from the dropdown → System updates the list of available flat types and filters for that town.*
2. *User selects a flat type → System updates the filtered result set.*
3. *User applies additional filters (storey range, floor area, flat model) → System narrows the displayed records accordingly.*

4.1.3 Functional Requirements

4.1.3.1: The system shall allow the user to select an HDB town as an input for analysis.
4.1.3.2: The system shall allow the user to select an HDB flat type as an analysis filter.
4.1.3.3: The system shall allow the user to apply additional available housing filters, including storey range, floor area, or flat model.

4.2 View Latest Price (Included, from 4.1 Select Housing Filters)

4.2.1 Description and Priority

After the user selects housing filters in UC-4.1, the system displays the most recent HDB resale transaction records (e.g. from the current year). Each entry shows the resale price and key property features such as flat type, number of bedrooms, town/area, etc. This gives the user an immediate snapshot of what similar flats have recently sold for.

***Priority**: High*

4.2.2 Stimulus/Response Sequences

1. *User confirms filter selections from UC-4.1 → System retrieves the most recent resale transaction records matching the selected criteria and displays them as property listing entries.*
2. *User scrolls through the listing entries → System loads additional records if available (pagination or infinite scroll).*
3. *User selects an individual listing entry → System expands or navigates to show the full transaction details for that record.*

4.2.3 Functional Requirements

4.2.3.1: The system shall retrieve and display the most recent HDB resale transaction records that match the user's selected filter criteria from UC-4.1.
4.2.3.2: Each listing entry shall display the resale price in SGD, flat type, town/area, storey range, floor area, remaining lease, and transaction period.
4.2.3.3: The system shall display the results in reverse chronological order, with the most recent transactions shown first.
4.2.3.4: The system shall limit the initial display to recent transactions (e.g. within the current calendar year) to give the user a current market snapshot.
4.2.3.5: The system shall allow the user to view the full details of a selected transaction record.

4.3 View Historical Data (Extend, from 4.1 Select Housing Filters)(End User)

4.3.1 Description and Priority

Retrieves and displays historical resale transaction records that match the user's selected search criteria.

***Priority**: High.*

4.3.2 Stimulus/Response Sequences

1. *User confirms search criteria → System queries the dataset and returns matching transaction records.*
2. *User views the results page → System displays the historical records in a readable list/table format.*

4.3.2 Functional Requirements

4.3.2.1: The system shall retrieve historical resale transaction records that satisfy the selected criteria.
4.3.2.2: The system shall display historical resale-price information for the selected criteria.

4.4 View Summary Statistics (Extend, from 4.1 Select Housing Filters) (End User)

4.4.1 Description and Priority

Allows the user to view summary statistics and historical resale-price trends based on the selected housing criteria from the Search for Latest House Prices use case.

***Priority**: Medium.*

4.4.2 Stimulus/Response Sequences

1. *User requests price analysis for the selected criteria → System computes summary statistics (e.g. mean, median resale price).*
2. *System renders the trend graph → User views price movement over time with each point's time period labeled.*

4.4.3 Functional Requirements

4.4.3.1: The system shall calculate summary price statistics from the selected historical transaction records.
4.4.3.2: The system shall display historical resale-price trends over time using a graphical representation.
4.4.3.3: The system shall display the time period represented by every historical price value shown on a graph.

4.5 View Fair Price Prediction (Extend, from 4.1 Select Housing Filters) (End User)

4.5.1 Description and Priority

Generates an estimated resale price for a set of user-entered property characteristics, using the trained OLS regression model ($\hat{Y} = x \cdot \beta$).

***Priority**: High.*

4.5.2 Stimulus/Response Sequences

1. *User enters property characteristics (town, flat type, storey range, flat model, floor area, lease commencement date) and clicks "Predict"*
2. *The system encodes the inputs into a feature vector and computes the predicted price.*
3. *User submits an input combination not represented in the training data (e.g. an unseen town/flat-type pairing) → System displays a reliability warning alongside the prediction.*

4.5.3 Functional Requirements

4.5.3.1: The system shall allow the user to input a town, flat type, storey range, flat model, floor area, and lease commencement date.
4.5.3.2: The system shall generate an estimated resale price for the entered property characteristics using the trained regression model.
4.5.3.3: The system shall warn the user when the submitted input combination was not represented in the training data.

4.6 View Prediction Breakdown (Included, from 4.5 View Fair Price Prediction)

4.6.1 Description and Priority

After the system generates an estimated resale price in UC-4.5, it automatically displays a breakdown showing how each input feature (town, flat type, floor area, etc) contributed to the final predicted price. This helps the user understand why the system estimated a particular value, making the prediction transparent and actionable rather than a black-box number.

***Priority**: High*

4.6.2 Stimulus/Response Sequences

1. *System generates a predicted resale price in UC-4.5 → System automatically computes and displays the contribution of each input feature to the predicted price.*
2. *User views the breakdown → System presents a visual representation (e.g. a table or bar chart) showing each feature's positive or negative contribution to the final estimated price.*

4.6.3 Functional Requirements

4.6.3.1: The system shall display the individual contribution of each input feature to the predicted resale price whenever a prediction is generated.
4.6.3.2: The system shall clearly indicate whether each feature's contribution increases or decreases the predicted price.
4.6.3.3: The system shall present the breakdown in a format understandable to a general user without statistical expertise.

4.7 Validate and Handle Error (included by 4.2 Select Housing Filters, maybe need to separate the errors

4.7.1 Description and Priority

Catches invalid or insufficient inputs before they reach the analysis or prediction engine, and communicates data/model limitations clearly to the user.

***Priority**: High.*

4.7.2 Stimulus/Response Sequences

1. *User submits filter or input values → System validates them before executing any query or prediction; invalid values are rejected with a specific error message.*
2. *User's search criteria return zero matching records → System displays a "no data found" message.*

3. *User's search criteria have too little historical data to support a reliable prediction → System displays an insufficient-data warning instead of a result.*

4.7.3 Functional Requirements

4.7.3.1: The system shall reject invalid filter values before executing an analysis.
4.7.3.2: The system shall inform the user when no historical transaction data matches the selected criteria.
4.7.3.3: The system shall inform the user when insufficient historical data prevents a requested prediction from being generated reliably.

4.8 View Model Diagnostics (Admin) add more detail regarding the breakdown

4.8.1 Description and Priority

Reports the trained model's goodness-of-fit metrics and shows how each input feature contributed to a given prediction.

***Priority**: Medium.*

4.8.2 Stimulus/Response Sequences

1. *User navigates to the diagnostics page → System displays R^2, Adjusted R^2 for the current model.*
2. *User requests a coefficient breakdown after receiving a prediction → System displays a table of each feature's coefficient and its contribution to the predicted price.*

4.8.3 Functional Requirements

4.8.3.1: The system shall report R^2, Adjusted R^2 for the trained model.
4.8.3.2: The system shall display the contribution of each input feature to a given prediction.

4.9 Manage Database (Admin)

4.9.1 Description and Priority

Allows an Admin to manually modify/check a new historical housing dataset and validate it before it is used for analysis or training, including converting text fields such as remaining lease into numeric form.

***Priority**: Medium.*

4.9.2 Stimulus/Response Sequences

1. *Authorised user modify/check a .csv or .xlsx file → System validates the updated file against the expected schema (required columns present, correct types).*
2. *System completes validation → System displays a confirmation with a row/column summary, or lists specific errors and missing columns if validation fails.*

4.9.3 Functional Requirements

4.9.3.1: The system shall allow an authorised user to import a supported historical housing dataset (.csv or .xlsx).
4.9.3.2: The system shall validate imported housing data before incorporating it into the analytical dataset, including parsing remaining_lease from text (e.g. "61 years 04 months") into a numeric value.

4.10 Update Price Database (API)

4.10.1 Description and Priority

Allows the system to periodically call the data.gov.sg API (HDB Resale Flat Prices dataset) to retrieve the newest transaction records and update the internal price database, so that all price information shown to End User reflects the latest available market data.

***Priority**:High.*

4.10.2 Stimulus/Response Sequences

1. *Scheduled job initiates a request to data.gov.sg API endpoint for new transaction records.*
2. *Data.gov.sg responds with latest dataset in JSON/CSV.*
3. *System checks for new records not already present in local database.*
4. *Valid new records are inserted into the price database; Invalid records flagged for Admin review.*
5. *The system updates the “last updated” timestamp shown to End User.*
6. *If the API call fails (timeout, rate limit), the system logs errors and flags for Admin.*

4.10.3 Functional Requirements

4.10.3.1: The system shall call the data.gov.sg API on a scheduled basis (daily) without requiring manual trigger
4.10.3.2: The system shall not overwrite historical price records, only append new transactions.
4.10.3.3: The system shall record the timestamp of last successful update.
4.10.3.4: The system shall flag the Admin if data retrieval fails.

4.11 Authorize Admin (Included by 4.8 View Model Diagnostics & 4.9 Manage Database)(Admin)

4.11.1 Description and Priority

Verifies the identity and role of an Admin before the system grants access to Admin-only functions. When an Admin attempts to use UC-4.8 or UC4.9, this use case checks for a valid authorized session. If none exists, it prompts for credentials, verifies them, and confirms the account holds the Admin role. This prevents End Users and unauthorized visitors from viewing model internals or modifying the housing dataset.

***Priority**: High.*

4.11.2 Stimulus/Response Sequences

1. *Admin requests a restricted function (UC-4.8 View Model Diagnostics or UC-4.9 Manage Database) → System checks for an active authenticated Admin session.*
2. *No active session exists → System displays a login prompt.*
3. *Admin submits username and password → System verifies the credentials and confirms the account holds the Admin role.*
4. *Credentials are valid → System establishes an authorized session and returns control to the requested function.*
5. *Credentials are invalid → System displays a generic error message and displays a login prompt again.*

4.11.3 Functional Requirements

4.11.3.1: The system shall require successful Admin authorization before granting access to UC-4.8 View Model Diagnostics or UC-4.9 Manage Database.
4.11.3.2: The system shall authorize an Admin by verifying the submitted username and password against stored Admin account records.
4.11.3.3: The system shall reject invalid credentials with a generic error message that does not reveal whether the username or the password was incorrect.
4.11.3.4: The system shall deny access to Admin-only pages, functions and data for unauthorized users and End Users, including requests made by direct URL.
4.11.3.5: Credential verification shall respond within two seconds (NFR 5.1.1).

5. Other Nonfunctional Requirements

5.1 Performance Requirements

Performance requirements ensure the system feels responsive during interactive use. Historical data retrieval and price prediction are the two operations users perform most frequently, so both have explicit timing targets under normal load; all other UI actions that don't trigger a computation are held to a tighter two-second bound.

5.1.1 Response Time

- *The system shall display requested historical housing-price information within three seconds under normal operating conditions.*
- *The system shall generate and display a forecast result within five seconds under normal operating conditions.*
- *User interface actions that do not require forecast computation shall respond within two seconds under normal operating conditions.*

5.1.2 Scalability and External Dependencies API

- *API calls to external services (data.gov.sg) should timeout after 10 seconds*

5.2 Safety Requirements

Because the system produces price estimates rather than certified valuations, the primary safety concern is preventing users from mistaking a statistical forecast for a guaranteed or official figure, particularly since resale-price decisions can carry significant financial consequences for buyers and sellers

5.2.1 Estimate Transparency

- *The system shall clearly state that future price values are statistical estimates and are not guaranteed market outcomes.*
- *The system shall not present forecasted results as official property valuations or financial advice.*

5.3 Security Requirements

The system shall clearly state that future price values are statistical estimates and are not guaranteed market outcomes. The system shall not present forecasted results as official property valuations or financial advice

5.3.1 Authentication and Data Protection

- *If user accounts are implemented, user passwords shall not be stored in plaintext.*
- *If administrator functionality is implemented, only authenticated administrators shall be permitted to modify or replace the system dataset.*

5.4 Software Quality Attributes

The system's primary audience is the general public rather than data professionals (see User Classes, §2.3), so usability and compatibility are prioritised over more specialised quality attributes. Reliability, maintainability, and testability requirements protect the integrity of historical data and keep the codebase verifiable as the team iterates.

5.4.1 Usability

- *A first-time user shall be able to perform a basic town price search without requiring technical knowledge of statistical modelling.*

5.4.2 Compatibility

- *The application shall support recent versions of Google Chrome, Apple Safari, and Microsoft Edge.*

5.4.3 Reliability

- *The system shall not modify original historical transaction values during analysis or forecasting.*

5.4.4 Maintainability

- *Data preprocessing, forecasting, and user-interface functionality shall be implemented as separate software components where practical.*

5.4.5 Testability

- *Each functional requirement shall be independently testable using defined input conditions and observable output.*

5.5 Business Rules

These rules govern how forecast results must be labelled and when the system is permitted to generate one, so that users are never misled about whether a figure is an actual historical sale or a model estimate

5.5.1 Forecast Scope and Labelling Forecast Horizons

Forecast Scope and Labelling Forecast horizons shall be limited to a maximum of ten years. Forecast results shall always be identified as estimates. Historical transaction values shall not be labelled as forecasted values, and forecasted values shall not be labelled as historical values.

5.5.2 Data Integrity and Product Scope

Only valid historical records shall be included in model training and price analysis. A forecast shall not be generated where the system determines that insufficient historical information exists for the selected criteria. Version 1.0 shall be limited to Singapore HDB resale housing.

6. Other Requirements

<Define any other requirements not covered elsewhere in the SRS. This might include database requirements, internationalization requirements, legal requirements, reuse objectives for the project, and so on. Add any new sections that are pertinent to the project.>

****

6.1 Data Requirements

- *Historical transaction records shall be stored in a structured tabular format.*
- *Each record used for analysis shall contain the fields required by the implemented forecasting model.*
- *Historical records shall contain, or allow derivation of, a transaction period/time index.*
- *The system shall preserve historical resale-price records without modifying their original values during forecasting.*

6.2 Forecast Output Requirements

Forecast output shall include, where applicable:

- *Selected town.*
- *Selected housing characteristics.*
- *Forecast horizon.*
- *Corresponding forecast year or period.*
- *Estimated resale price.*
- *A clear indication that the displayed value is forecasted.*

Appendix A: Glossary

<Define all the terms necessary to properly interpret the SRS, including acronyms and abbreviations. You may wish to build a separate glossary that spans multiple projects or the entire organization, and just include terms specific to a single project in each SRS.>

****
it

Term	***Description***	***Main Attributes***	***Relationships***
Town	*Singapore HDB residential town associated with a resale transaction.*	*townName*	*Has many Transactions.*
Flat	*Housing characteristics represented by a transaction.*	*flatType, storeyRange, floorAreaSqm, flatModel, leaseCommenceDate, remainingLease*	*Represented in a Transaction.*
Transaction	*Historical record of an HDB resale transaction.*	*transactionID, transactionPeriod, resalePrice, town, flatType*	*Belongs to one Town and describes one resale transaction.*

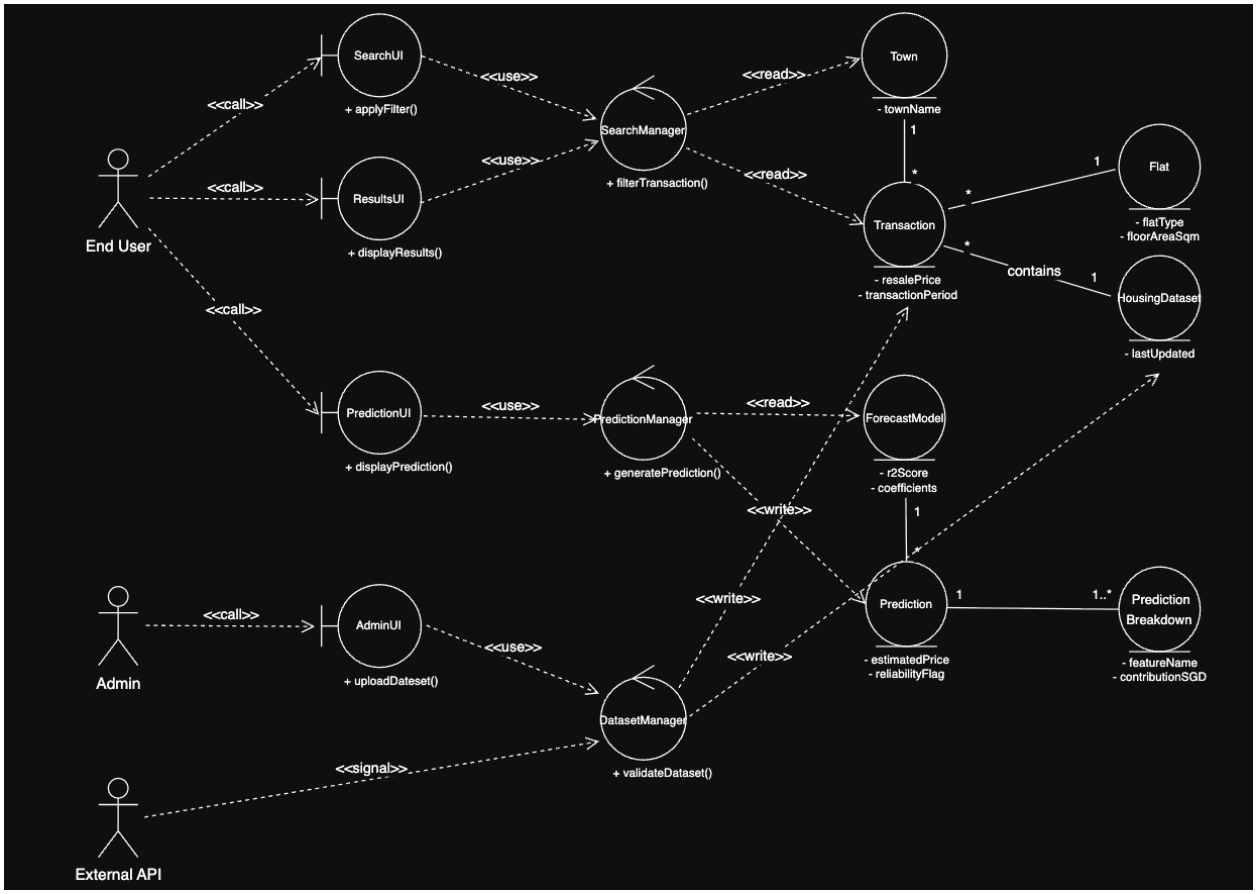
Historical Price	*Actual resale-price observation obtained from historical transaction data.*	*period, resalePrice*	*Derived directly from a Transaction.*
Price Trend	*Aggregated representation of resale-price changes over time.*	*period, statisticValue, town, flatType*	*Derived from multiple Transactions.*
Forecast Request	*Criteria selected by the user to request future price estimates.*	*town, flatType, additionalFilters, forecastHorizon*	*Produces a Forecast.*
Forecast	*Estimated future HDB resale-price output.*	*forecastYear, estimatedPrice, town, flatType*	*Generated from historical data using a Forecast Model.*
Forecast Model	*Statistical model responsible for producing future resale-price estimates.*	*modelType, trainingPeriod, modelParameters, evaluationMetrics*	*Uses historical Transactions to generate Forecasts.*
Housing Dataset	*Collection of historical HDB resale transaction records.*	*datasetID, source, version, lastUpdated*	*Contains multiple Transactions.*
Filter Criteria	*Conditions selected by the user to narrow the analysed dataset.*	*town, flatType, storeyRange, floorArea, flatModel*	*Applied to Transactions.*
Transaction Period	*Time associated with a historical resale transaction.*	*year and/or month*	*Used to construct historical Price Trends and future forecast periods.*

Appendix B: Use Case Diagram & Descriptions

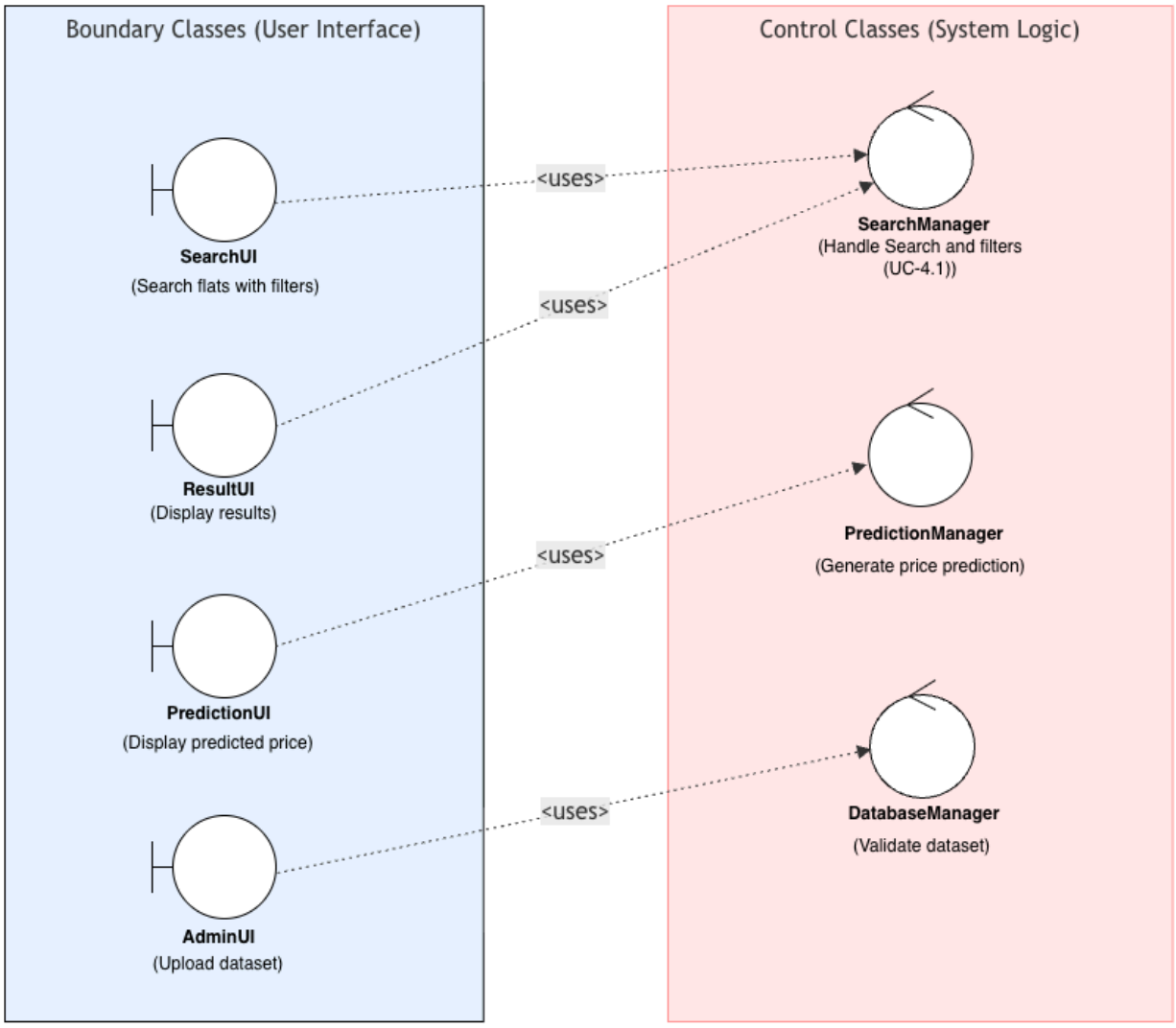
(paste the use case diagram and descriptions all into here)

Appendix C: Analysis Models

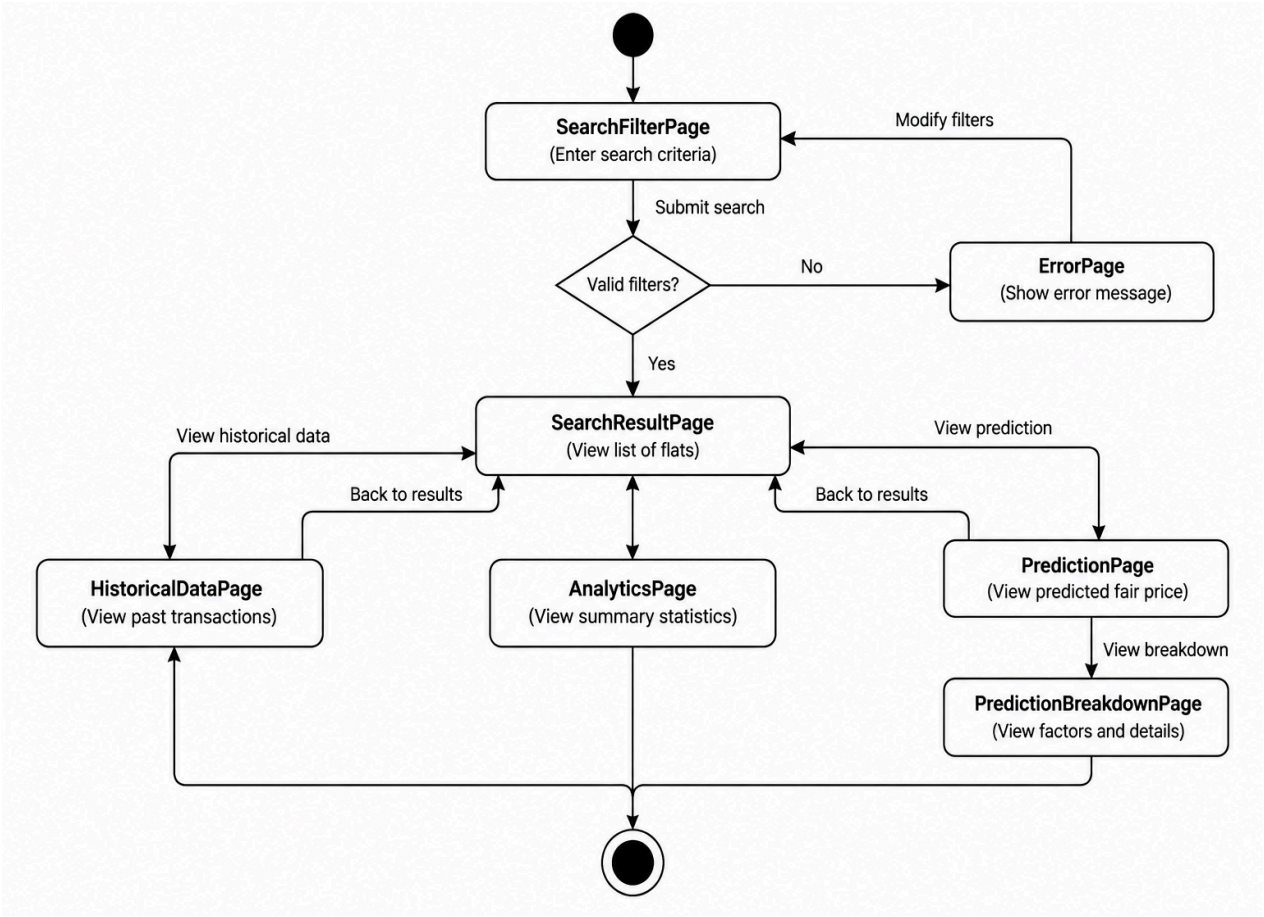
<Optionally, include any pertinent analysis models, such as data flow diagrams, class diagrams, state-transition diagrams, or entity-relationship diagrams.>



Class Diagram



Key Boundary Classes and Control Classes



Dialogue Map

Appendix D: To Be Determined List

<Collect a numbered list of the TBD (to be determined) references that remain in the SRS so they can be tracked to closure.>

******

TBD-01: Exact HDB resale dataset source, version, and update procedure.

TBD-02: Final frontend framework, if any.

TBD-03: Final database/storage technology.

TBD-04: Final regression/forecasting technique after model evaluation.

TBD-05: Whether user accounts and administrator functionality will be implemented.

TBD-06: Whether dataset upload will be exposed as a user-facing function. (only by Admin)

TBD-07: Exact thresholds used to determine when historical data is insufficient for forecasting.(outliers?)

Source: http://www.frontiernet.net/~kwiegers/process_assets/srs_template.doc

Use Cases

for

<Singapore Zone-Based Real Estate Linear Regression Predictor>

Version 1.0 approved

Prepared by <author>

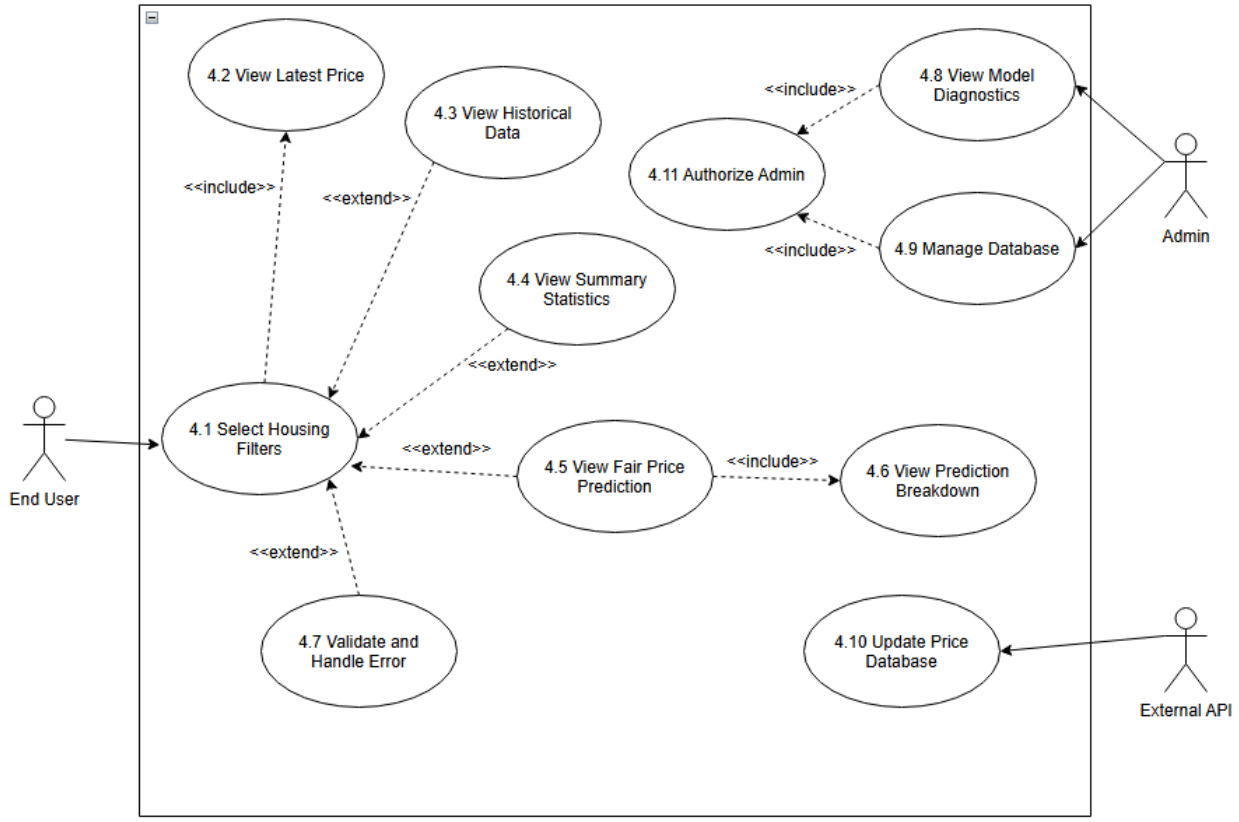
<Group 2>

<date created>

Revision History

Name	Date	Reason For Changes	Version

1. Use Case Diagram



Simplified Use Case Logic

Visitor

- Search price by area/district/flat type
- View historical price trends
- View comparison across towns
- *(extend)* Register for alerts

Admin

- Validate/clean incoming data
- Manage area/district metadata
- Publish insights or reports

External Data System

- Push new transaction data → *(include)* Update price database

End User connects directly to 4.2 Search, 4.3 View Historical Data, and 4.6 View Price Prediction, these are the entry points a person can trigger.

4.4 View Price Analysis and **4.5 View Area Comparison** have no direct actor line, they're only reachable via `<<extend>>` from 4.2 Search, matching your confirmation that they're optional add-ons.

4.8 Validate and Handle Error (gray, no actor) is pulled in automatically via `<<include>>` from both 4.2 Search and 4.6 View Price Prediction, it runs silently whenever either of those fires.

Admin connects to 4.7 View Model Diagnostics and 4.9 Manage Dataset.

External API (data.gov.sg) connects to 4.10 Update Price Database, which sits inside the system boundary since it's a system-internal process, just externally triggered.

2. NEW USE-CASE DESCRIPTION

Use Case ID:	UC-4.1		
Use Case Name:	Select Housing Filters		
Created By:	Yan Xi	Last Updated By:	Justin
Date Created:	14/8/2026	Date Last Updated:	7/9/2026

Actor:	End User (Initiating Actor)
Description:	The user narrows the HDB resale transaction dataset by selecting a town, flat type, and optional additional housing characteristics. This is the primary entry point for all downstream analysis features.
Preconditions:	1. The system is available and accessible via a web browser. 2. The HDB resale transaction dataset is loaded and accessible to the system.
Postconditions:	1. The system has recorded the user's selected filter criteria. 2. The system displays the most recent matching transaction records via UC-4.2 View Latest Price.
Priority:	High
Frequency of Use:	High, used every time an End User begins interacting with the system.
Flow of Events:	1. The user navigates to the home/search page. 2. The user selects a town from the dropdown. 3. The system updates the available flat types and other filter options for the selected town. 4. The user selects a flat type. 5. The user optionally applies additional filters (storey range, floor area, flat model). 6. The system records the selected filter criteria. 7. The system uses the included use case UC-4.2 View Latest Price to display the most recent matching transactions.
Alternative Flows:	UC-4.1.AC.1: User changes the selected town → System refreshes the available flat types and resets dependent filters. UC-4.1.AC.2: User changes or removes an additional filter → System updates the applied filter criteria accordingly. UC-4.1.AC.3: User leaves an optional filter unspecified → System does not apply that filter.
Exceptions:	UC-4.1.EX.1: The dataset is unavailable or cannot be loaded → System displays an error message informing the user that the service is temporarily unavailable.

Includes:	UC-4.2 View Latest Price (always displays results after filters are applied).
Special Requirements:	1. Filter options shall update dynamically based on the user's prior selections. 2. The filtering operation shall respond within two seconds (NFR 5.1.1).
Assumptions:	1. The dataset contains valid HDB resale transaction records. 2. Town, flat type, storey range, floor area, and flat model are available as filtering attributes.
Notes and Issues:	This use case is the single entry point from which all other End User use cases are reached, either via <<include>> (4.2) or <<extend>> (4.3, 4.4, 4.5, 4.7).

Use Case ID:	UC-4.2		
Use Case Name:	View Latest Price		
Created By:	Justin	Last Updated By:	Justin
Date Created:	29/8/2026	Date Last Updated:	8/9/2026

Actor:	End User (via UC-4.1)
Description:	*After the user selects housing filters in UC-4.1, the system automatically displays the most recent HDB resale transaction records (e.g. from the current year) as browsable property listing entries. Each entry shows the resale price and key property features, giving the user an immediate snapshot of what similar flats have recently sold for.*
Preconditions:	1. UC-4.1 Select Housing Filters has been completed with at least a town selected. 2. The historical transaction dataset is loaded and accessible.
Postconditions:	1. The most recent HDB resale transaction records matching the selected criteria are displayed as listing entries. 2. Each entry shows the resale price and key property features.
Priority:	High
Frequency of Use:	High, runs automatically every time UC-4.1 completes (<<include>>).
Flow of Events:	1. UC-4.1 passes the selected filter criteria to this use case. 2. The system retrieves the most recent resale transaction records matching the selected criteria. 3. The system displays the results as property listing entries in reverse chronological order. 4. Each entry shows resale price in SGD, flat type, town/area, storey range, floor area, remaining lease, and transaction period. 5. The user scrolls through the listing entries.

Alternative Flows:	UC-4.2.AC.1: User selects an individual listing entry → System expands or navigates to show the full transaction details for that record. UC-4.2.AC.2: More records are available than the initial display → System loads additional records via pagination or infinite scroll. UC-4.2.AC.3: User changes a filter in UC-4.1 → System re-retrieves and refreshes the listing entries with the updated criteria.
Exceptions:	UC-4.2.EX.1: No recent transaction records match the selected criteria → System displays a "No recent transactions found" message and suggests the user broaden their filters or view historical data instead.
Includes:	Included by UC-4.1 Select Housing Filters.
Special Requirements:	1. The initial display shall be limited to recent transactions (e.g. within the current calendar year). 2. Results shall be displayed within three seconds under normal operating conditions (NFR 5.1.1).
Assumptions:	1. The dataset contains recent transaction records with complete information for display. 2. The selected filter criteria from UC-4.1 are valid and available.
Notes and Issues:	This is the default view after filtering, the user sees what similar flats have recently sold for before optionally exploring historical data, statistics, or predictions.

Use Case ID:	UC-4.3		
Use Case Name:	View Historical Data		
Created By:	Calvin	Last Updated By:	Justin
Date Created:	29/8/2026	Date Last Updated:	8/9/2026

Actor:	End User (via UC-4.1)
Description:	*The user optionally chooses to view all historical HDB resale transaction records (not limited to the current year) that match the housing criteria selected in UC-4.1. The system retrieves and displays the full historical records in a readable table.*
Preconditions:	1. UC-4.1 Select Housing Filters has been completed with at least a town selected.

	2. The historical transaction dataset is loaded and accessible to the system.
Postconditions:	1. The user can view historical HDB resale transaction records matching the selected criteria. 2. The records are displayed in a readable table containing relevant transaction information.
Priority:	High
Frequency of Use:	Frequent, whenever the user wants to explore the full transaction history beyond the recent listings shown in UC-4.2.
Flow of Events:	1. The user clicks the "View Historical Data" option from the results page. 2. The system retrieves all historical transaction records matching the selected criteria from UC-4.1. 3. The system displays the records in a table showing town, flat type, floor area, storey range, resale price, remaining lease, and transaction period. 4. The user browses the historical records.
Alternative Flows:	UC-4.3.AC.1: User changes one or more search filters → System re-queries and updates the displayed historical records based on the new criteria without leaving the page.
Exceptions:	UC-4.3.EX.1: No records match the selected criteria → System displays a "No matching records found" message instead of an empty table.
Includes:	-
Special Requirements:	1. Historical records shall be displayed within three seconds under normal operating conditions (NFR 5.1.1). 2. The displayed records shall clearly identify the relevant transaction information.
Assumptions:	1. The historical transaction dataset contains the required information for the displayed records. 2. The selected search criteria from UC-4.1 are valid and available to UC-4.3.
Notes and Issues:	Extends UC-4.1 (<<extend>>). Pagination should be implemented if the number of matching records is large.

Use Case ID:	UC-4.4		
Use Case Name:	View Summary Statistics		
Created By:	Yan Xi	Last Updated By:	Justin
Date Created:	14/8/2026	Date Last Updated:	7/9/2026

Actor:	End User (via UC-4.1)
Description:	The user optionally chooses to view summary statistics and historical resale-price trends based on the housing criteria selected in UC-4.1. The system calculates statistical summaries (e.g. mean, median resale price) and displays a time-series graph showing price movement over time.
Preconditions:	1. UC-4.1 Select Housing Filters has been completed with at least the required search criteria selected. 2. Historical transaction records matching the selected criteria are available.
Postconditions:	1. The system displays the calculated summary price statistics for the selected criteria. 2. A time-series graph showing historical resale-price trends is displayed with each data point labelled by its corresponding time period.
Priority:	Medium
Frequency of Use:	Frequent, whenever the user wants to analyse historical price trends for a selected set of housing criteria.
Flow of Events:	1. The user clicks the "View Summary Analysis" option from the results page. 2. The system retrieves the corresponding historical transaction records. 3. The system calculates summary price statistics such as mean and median resale price. 4. The system generates a time-series graph using the historical resale-price data. 5. The system labels each historical price value with its corresponding time period. 6. The user views the summary statistics and price trend graph.
Alternative Flows:	UC-4.4.AC.1: User changes the selected search criteria in UC-4.1 → System recalculates the statistics and updates the graph based on the new criteria. UC-4.4.AC.2: User selects a different time range or analysis option → System updates the displayed analysis accordingly.
Exceptions:	UC-4.4.EX.1: No records match the selected criteria → System informs the user that no matching records were found. UC-4.4.EX.2: The dataset cannot be accessed → System displays an appropriate error message.
Includes:	-
Special Requirements:	1. The graph must clearly show resale-price movement over time. 2. Every displayed historical price value must have its corresponding time period identified. 3. The graph and statistics should update within three seconds when the selection changes (NFR 5.1.1).

Assumptions:	1. Historical transaction records contain valid resale-price and time-period information. 2. The system has sufficient historical data to generate meaningful statistics and trends.
Notes and Issues:	Extends UC-4.1 (<<extend>>). The summary analysis may be used as supporting information for downstream price prediction.

Use Case ID:	UC-4.5		
Use Case Name:	View Fair Price Prediction		
Created By:	Xavier	Last Updated By:	Justin
Date Created:	14/8/2026	Date Last Updated:	8/9/2026

Actor:	End User (via UC-4.1)
Description:	The user optionally chooses to request an estimated resale price for the selected property characteristics. The system converts the inputs into the required feature representation, applies the trained OLS regression model, generates the predicted resale price, and automatically displays the prediction breakdown via UC-4.6.
Preconditions:	1. UC-4.1 Select Housing Filters has been completed. 2. A trained OLS regression model is available and loaded by the system. 3. The required property input fields are available to the user.
Postconditions:	1. The estimated HDB resale price is displayed to the user. 2. The prediction breakdown is displayed via UC-4.6. 3. If the submitted input combination was not represented in the training data, a reliability warning is displayed alongside the prediction.
Priority:	High
Frequency of Use:	Frequent, whenever a user wants to estimate the resale price of an HDB property.
Flow of Events:	1. The user clicks the "View Prediction" option from the results page. 2. The system pre-fills available fields from the UC-4.1 filter selections (town, flat type, etc.). 3. The user confirms or adjusts the property characteristics (town, flat type, storey range, flat model, floor area, lease commencement date). 4. The user clicks "Predict". 5. The system validates the submitted inputs. 6. The system encodes the property characteristics into the required feature vector. 7. The system applies the trained OLS regression model to the feature vector.

	8. The system computes the estimated resale price. 9. The system displays the predicted resale price to the user. 10. The system uses the included use case UC-4.6 View Prediction Breakdown to display the feature contributions.
Alternative Flows:	UC-4.5.AC.1: User modifies one or more property characteristics and submits another prediction → System recalculates and displays the updated predicted resale price and breakdown. UC-4.5.AC.2: The submitted property combination was not represented in the training data → System still generates the prediction but displays a reliability warning alongside it.
Exceptions:	UC-4.5.EX.1: One or more required input fields are missing → System does not generate a prediction and prompts the user to complete the required fields. UC-4.5.EX.2: An input value is invalid or outside the accepted format/range → System rejects the input and asks the user to correct it.
Includes:	UC-4.6 View Prediction Breakdown (always displays after a prediction is generated).
Special Requirements:	1. The prediction shall be generated using the trained OLS regression model. 2. A reliability warning must be clearly shown when the submitted input combination was not represented in the training data. 3. The prediction shall be generated and displayed within five seconds under normal operating conditions (NFR 5.1.1).
Assumptions:	1. The trained model and its required feature encoding are correctly configured and available to the application. 2. The filter criteria from UC-4.1 can be used to pre-fill prediction inputs.
Notes and Issues:	Extends UC-4.1 (<<extend>>). The reliability warning indicates limited representation in the training data and should not be interpreted as a prediction failure.

Use Case ID:	UC-4.6		
Use Case Name:	View Prediction Breakdown		
Created By:	Justin	Last Updated By:	Justin
Date Created:	9/9//2026	Date Last Updated:	9/9/2026

Actor:	End User (via UC-4.5)
Description:	*After the system generates an estimated resale price in UC-4.5, it automatically displays a breakdown showing how each input feature (town, flat type, storey range, flat model, floor area, lease commencement date) contributed to the final predicted price. This*

	makes the prediction transparent and actionable rather than a black-box number.
Preconditions:	1. UC-4.5 View Fair Price Prediction has successfully generated an estimated resale price. 2. The trained model's feature coefficients are available to the system.
Postconditions:	1. The contribution of each input feature to the predicted resale price is displayed to the user. 2. Each contribution is clearly labelled as a positive or negative influence on the price.
Priority:	High
Frequency of Use:	Every time a prediction is generated, runs automatically via <<include>> from UC-4.5.
Flow of Events:	1. UC-4.5 generates the estimated resale price and passes the encoded feature vector and model coefficients to this use case. 2. The system calculates the individual contribution of each feature by multiplying the feature's encoded value by its corresponding model coefficient. 3. The system displays the breakdown to the user, showing each feature's name, its value as entered, and its contribution in SGD (positive or negative) to the final predicted price. 4. The user views the breakdown alongside the predicted price.
Alternative Flows:	UC-4.6.AC.1: User modifies property characteristics in UC-4.5 and submits a new prediction → System recalculates and updates the breakdown to reflect the new feature contributions.
Exceptions:	UC-4.6.EX.1: The model coefficients are unavailable or corrupted → System displays the predicted price from UC-4.5 without the breakdown and informs the user that the feature breakdown is temporarily unavailable.
Includes:	Included by UC-4.5 View Fair Price Prediction.
Special Requirements:	1. The breakdown must be presented in plain language understandable by a general user, no raw coefficient values or statistical jargon. 2. Contributions shall be displayed in SGD (e.g. "+$25,000 for high floor" or "−$18,000 for shorter remaining lease").
Assumptions:	1. The trained OLS regression model's coefficients are stored and accessible at prediction time. 2. The linear nature of the model allows each feature's contribution to be calculated independently as the product of its coefficient and encoded value.
Notes and Issues:	This is distinct from UC-4.8 View Model Diagnostics (Admin), which reports technical metrics like R^2 and RMSE. This breakdown is a user-facing explanation of a specific prediction, not a model evaluation tool.

Use Case ID:	UC-4.7		
Use Case Name:	Validate and Handle Error		
Created By:	Triton	Last Updated By:	Justin
Date Created:	29/8/2026	Date Last Updated:	9/9/2026

Actor:	System (triggered internally; no direct actor)
Description:	Validates user-submitted filter and prediction inputs before they reach the analysis or prediction engine, and communicates data or model limitations back to the user clearly. This use case extends UC-4.1 and is triggered when invalid or problematic input is detected.
Preconditions:	A triggering use case (UC-4.1 Select Housing Filters or UC-4.5 View Fair Price Prediction) has been invoked with user-submitted input.
Postconditions:	Either: (a) validated input is passed back to the triggering use case for normal processing, or (b) the user sees a specific, actionable error or warning message and processing halts.
Priority:	High
Frequency of Use:	Triggered whenever invalid input, missing data, or insufficient historical data is detected during UC-4.1 or UC-4.5.
Flow of Events:	1. The triggering use case submits input values to the validation routine. 2. The system checks each value against expected type, format, and range rules. 3. If all values are valid, the system returns control to the triggering use case. 4. If a search returns zero matching records, the system displays a "no data found" message. 5. If a prediction request has too little supporting historical data, the system displays an insufficient-data warning instead of a prediction.
Alternative Flows:	-
Exceptions:	UC-4.7.EX.1: A submitted value fails format or range validation → System rejects the request with a field-specific error message. UC-4.7.EX.2: Zero records match the search criteria → "No data found" message shown. UC-4.7.EX.3: Insufficient historical data for a reliable prediction → Insufficient-data warning shown instead of a result.
Includes:	Extended from UC-4.1 (Select Housing Filters). Also invoked by UC-4.5 (View Fair Price Prediction).
Special Requirements:	Messages must be specific enough for the user to correct input or understand the limitation, not a generic "an error occurred."

Assumptions:	Thresholds for "insufficient historical data" are configurable and will be finalised by the team (TBD-07).
Notes and Issues:	Consider whether validation errors for filtering (UC-4.1) and prediction (UC-4.5) should be handled as separate sub-flows within this use case for clarity.

Use Case ID:	UC-4.8		
Use Case Name:	View Model Diagnostics		
Created By:	Justin	Last Updated By:	Justin
Date Created:	9/9/2026	Date Last Updated:	9/9/2026

Actor:	Admin (Initiating Actor)
Description:	*The Admin views the trained regression model's technical performance metrics, including goodness-of-fit statistics and the full coefficient table. This allows the Admin to assess whether the model is performing adequately and to identify features with unexpected or disproportionate influence on predictions.*
Preconditions:	1. A trained OLS regression model is available in the system.
Postconditions:	1. The system displays the model's R^2 and Adjusted R^2 values. 2. The system displays the full table of feature coefficients and their statistical significance.
Priority:	Medium
Frequency of Use:	Occasional, after a new model is trained or when the Admin wants to audit model performance.
Flow of Events:	1. The system invokes UC-4.11 Authorize Admin. 2. The Admin navigates to the model diagnostics page. 3. The system retrieves the current trained model's metadata. 4. The system displays R^2, Adjusted R^2, and any other available goodness-of-fit metrics. 5. The system displays a table of all feature coefficients, showing each feature name and its corresponding weight. 6. The Admin reviews the diagnostics.
Alternative Flows:	UC-4.8.AC.1: Admin selects a specific prediction to inspect → System displays the coefficient breakdown for that specific prediction alongside the global model metrics.
Exceptions:	UC-4.8.EX.1: No trained model is available → System informs the Admin that model diagnostics cannot be displayed until a model has been trained.
Includes:	UC-4.11 Authorize Admin
Special Requirements:	1. Diagnostics must include R^2 and Adjusted R^2 at minimum.

	2. The coefficient table must show every feature used by the model.
Assumptions:	The trained model's metrics and coefficients are stored and retrievable by the system.
Notes and Issues:	This is a technical, Admin-facing view. The End User equivalent is UC-4.6 View Prediction Breakdown, which presents feature contributions in plain SGD terms without statistical jargon.

Use Case ID:	UC-4.9		
Use Case Name:	Manage Database		
Created By:	Justin	Last Updated By:	Justin
Date Created:	9/9/2026	Date Last Updated:	9/9/2026

Actor:	Admin (Initiating Actor)
Description:	*The Admin manually imports, modifies, or checks a historical HDB resale dataset. The system validates the uploaded or modified file against the predefined HDB dataset schema, checking that all mandatory columns are present and correctly formatted, before incorporating the data into the analytical dataset.*
Preconditions:	1. The Admin is authenticated and authorised. 2. The dataset file is in .csv or .xlsx format.
Postconditions:	1. The validated dataset is stored in the system and available for analysis and model training. 2. The system confirms successful ingestion and displays a summary (row count, column names).
Priority:	Medium
Frequency of Use:	Occasional, whenever a new or updated dataset needs to be loaded or checked manually.
Flow of Events:	1. The system invokes UC-4.11 Authorize Admin. 2. The Admin navigates to the dataset management page. 3. The Admin selects a .csv or .xlsx file from their local machine. 4. The Admin clicks "Upload". 5. The system reads the file and validates that mandatory columns are present (town, flat_type, storey_range, flat_model, floor_area_sqm, lease_commence_date, etc). 6. The system parses text fields such as remaining_lease (e.g. "61 years 04 months") into numeric values. 7. The system confirms successful upload and displays a summary of the ingested data.

Alternative Flows:	UC-4.9.AC.1: Admin uploads a file with extra columns beyond the mandatory ones → System accepts the file but ignores non-required columns and notifies the Admin.
Exceptions:	UC-4.9.EX.1: File is not .csv or .xlsx → System rejects the upload and displays an error message specifying accepted formats. UC-4.9.EX.2: File is missing one or more mandatory columns → System rejects the upload and lists the missing columns. UC-4.9.EX.3: File contains no data rows → System rejects and notifies the Admin.
Includes:	UC-4.11 Authorize Admin
Special Requirements:	File size should not exceed a configurable limit (e.g. 50 MB).
Assumptions:	The uploaded file follows the official HDB tabular schema.
Notes and Issues:	Consider supporting drag-and-drop upload in a future version.

Use Case ID:	UC-4.10		
Use Case Name:	Update Price Database		
Created By:	Triton	Last Updated By:	Justin
Date Created:	29/8/2026	Date Last Updated:	9/9/2026

Actor:	1. Scheduler / timed system job (Primary Actor) 2. data.gov.sg / External API (Secondary Actor)
Description:	On a daily schedule, the system calls the data.gov.sg HDB Resale Flat Prices API to retrieve newly published transaction records and appends them to the internal price database, so that price information shown to End Users reflects the latest available market data.
Preconditions:	1. The system has network access to the data.gov.sg API. 2. The internal price database is available for writes.
Postconditions:	1. New, valid transaction records are appended to the price database. 2. The "last updated" timestamp shown to End Users is refreshed. 3. Invalid or failed records are flagged for Admin review.
Priority:	High
Frequency of Use:	Automatic, scheduled daily.
Flow of Events:	1. The Scheduler triggers the update job at the configured time. 2. The system requests the latest transaction records from the data.gov.sg API endpoint. 3. data.gov.sg responds with the dataset in JSON/CSV.

	4. The system compares incoming records against existing ones to identify new transactions. 5. The system validates and appends new, valid records (existing historical records are never overwritten). 6. The system updates the "last updated" timestamp.
Alternative Flows:	-
Exceptions:	UC-4.10.EX.1: API call times out or is rate-limited (>10 seconds per NFR 5.1.2) → System logs the error, flags it for Admin, and leaves the existing database unchanged. UC-4.10.EX.2: Returned records fail schema validation → System flags the specific invalid records for Admin review without inserting them.
Includes:	-
Special Requirements:	1. External API calls shall time out after 10 seconds (NFR 5.1.2). 2. This use case shall never modify or overwrite existing historical price records (NFR 5.4.3 / Business Rule 5.5.2).
Assumptions:	The data.gov.sg API schema remains stable and compatible with the internal schema (TBD-01).
Notes and Issues:	Consider whether Admin should also be able to manually trigger this update on demand, in addition to the scheduled run.

Use Case ID:	UC-4.11		
Use Case Name:	Authorize Admin		
Created By:	Triton	Last Updated By:	Triton
Date Created:	14/10/2026	Date Last Updated:	14/10/2026

Actor:	Admin (Initiating Actor)
Description:	Verifies that a user attempting to access an Admin-only function is an authorized Admin. The system checks for an existing session, prompts for credentials if needed, verifies them and the Admin role, and then returns control to the use case that invoked it (UC-4.8 View Model Diagnostics or UC-4.9 Manage Database).
Preconditions:	1. The system is available and accessible via a web browser. 2. At least one Admin account has been provisioned in the system. 3. The Admin has attempted to access UC-4.8 View Model Diagnostics or UC-4.9 Manage Database.
Postconditions:	1. On success: an authorized Admin session is active and control returns to the invoking use case.

	2. On failure: access is denied, the invoking use case is not executed.
Priority:	High
Frequency of Use:	Occasional. It runs each time an Admin starts a session or accesses UC-4.8 View Model Diagnostics or UC-4.9 Manage Database without a valid session.
Flow of Events:	1. The Admin selects a restricted function, which invokes this use case. 2. The system checks whether a valid authorized Admin session exists. 3. If none exists, the system displays the login form. 4. The Admin enters a username and password and submits. 5. The system verifies the credentials against the stored Admin account records. 6. The system verifies that the account has the Admin role and is not locked. 7. The system establishes an authorized session. 8. The system returns control to the invoking use case.
Alternative Flows:	UC-4.11.AC.1: Admin chooses to log out → System ends the sessions and requires re-authorization for any further Admin function.
Exceptions:	UC-4.11.EX1: Username or Password is incorrect → System displays a generic “Invalid credentials” message, and re-prompts. UC-4.11.EX2: Username or Password field is left empty → System prompts the Admin to complete both fields.
Includes:	Included by UC-4.8 View Model Diagnostics & UC-4.9 Manage Database.
Special Requirements:	1. Credential verification shall respond within two seconds (NFR 5.1.1).
Assumptions:	1. Admin accounts are provisioned in advance, and there is no self-registration.
Notes and Issues:	1. Password reset and multi-factor authorization are out of scope for this Version 1.0.